

Singly Linked Lists

Representation, Traversal, Searching, Insertion and Deletion

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Why Do We Need Linked Lists?

Arrays are useful, but they have some limitations:

- Elements are stored in consecutive memory locations.
- The size is generally decided in advance.
- Increasing or decreasing the structure dynamically is difficult.
- Insertions and deletions may require shifting elements.

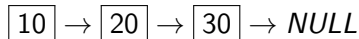
A linked list provides a more flexible way to store a sequence of elements.

Array



1000 1004 1008 1012

Linked List



What is a Singly Linked List?

A **singly linked list** is a linear data structure made up of nodes.

Each node contains two parts:

- **DATA**: stores the actual information.
- **NEXT**: stores the address of the next node.

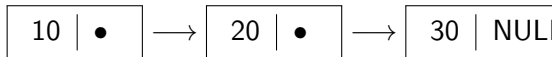
The address of the first node is stored in a pointer usually called:

START

The last node contains:

NEXT = NULL

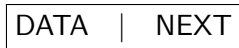
START



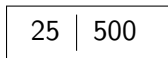
Traversal takes place only in the forward direction.

Anatomy of a Node

A node contains:



For example:

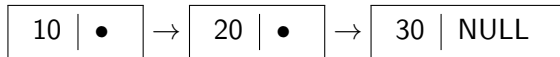


means:

- DATA = 25
- NEXT = address 500

The next node can be located anywhere in memory.

Logical View



Physical Memory

100 :

10

700 :

30

Linked List in Memory

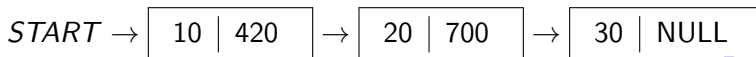
Consider the following memory representation:

Address	DATA	NEXT
100	10	420
420	20	700
700	30	NULL

If:

$$START = 100$$

then the logical linked list is:



START and NULL

START

- Stores the address of the first node.
- Used as the entry point into the list.

If:

$START = NULL$

then the linked list is empty.

NULL

- Indicates that no next node exists.
- Marks the end of the linked list.

Non-empty List

$START \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow NULL$

Empty List

$START \rightarrow NULL$

Arrays vs Linked Lists

Array	Linked List
Contiguous memory	Non-contiguous memory
Fixed declared capacity	Dynamic allocation
Direct/random access	Sequential access
Insertion may require shifting	Pointer changes are used
No pointer overhead	Extra pointer field required
Indexing is easy	Traversal is required

Linked List = Flexibility + Pointer Management

Representing a Node in C

```
struct Node
{
    int data;
    struct Node *next;
};
```

data

- Stores the actual value.

next

- Stores the address of another node.

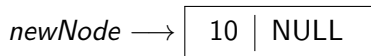
<i>data</i> <i>next</i>

Self-referential structure

Creating a Node Dynamically

```
1 struct Node *newNode;  
2  
3 newNode = (struct Node *)  
4         malloc(sizeof(struct Node));  
5  
6 newNode->data = 10;  
7 newNode->next = NULL;
```

This creates:



Important

Whenever `malloc()` is used, always check whether memory allocation was successful in robust

Creating a Small Linked List

Suppose we create three nodes containing:

10, 20, 30

The required links are:

10.*next* $\rightarrow$ 20

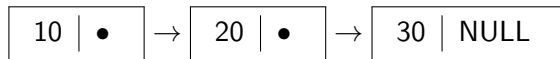
20.*next* $\rightarrow$ 30

30.*next* $\rightarrow$ *NULL*

Finally:

START $\rightarrow$ 10

START



Traversing a Singly Linked List

Traversal means visiting each node one by one.

We use a temporary pointer:

$$PTR = START$$

Then repeatedly:

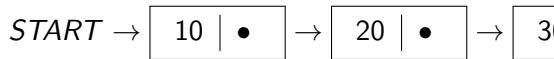
$$PTR = PTR \rightarrow next$$

Traversal continues until:

$$PTR = NULL$$

Typical uses:

- Display all nodes.
- Count nodes.
- Search for a value.



$$PTR \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow NULL$$

Traversal Algorithm

Algorithm

- 1 Set $PTR = START$.
- 2 Repeat while $PTR \neq NULL$.
- 3 Process $PTR \rightarrow DATA$.
- 4 Move to the next node:

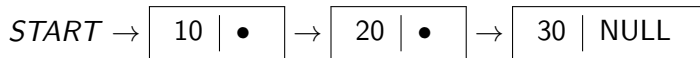
$$PTR = PTR \rightarrow NEXT$$

- 5 Stop when $PTR = NULL$.

$START \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow NULL$

Traversal: Dry Run

Initial List



Step	PTR points to	Output
1	10	10
2	20	20
3	30	30
4	NULL	Stop

Final output:

10	20	30
----	----	----

Traversal in C

```
void display(struct Node *start)
{
    struct Node *ptr = start;

    while (ptr != NULL)
    {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
}
```

Time complexity:

$$O(n)$$

because every node may have to be visited.

Counting Nodes

```
int countNodes(struct Node *start)
{
    int count = 0;
    struct Node *ptr = start;

    while (ptr != NULL)
    {
        count++;
        ptr = ptr->next;
    }

    return count;
}
```

10 → 20 → 30 → 40 → *NULL*

Searching in a Singly Linked List

To search for a value:

- 1 Start from the first node.
- 2 Compare the current node's data with the required value.
- 3 If equal, search is successful.
- 4 Otherwise move to the next node.
- 5 Stop when the value is found or NULL is reached.

Worst-case complexity:

$$O(n)$$

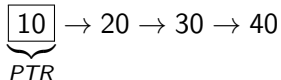
Search for **30**:

10 → 20 → 30 → 40 → *NULL*

Three nodes examined.

Searching for 30: Pointer Movement

Step 1



$$10 \neq 30$$

Step 2



$$20 \neq 30$$

Searching in C

```
struct Node *search(struct Node *start, int value)
{
    struct Node *ptr = start;

    while (ptr != NULL)
    {
        if (ptr->data == value)
            return ptr;

        ptr = ptr->next;
    }

    return NULL;
}
```

Insertion in a Singly Linked List

Common insertion cases are:

- 1 Insertion at the beginning.
- 2 Insertion at the end.
- 3 Insertion after a given node.
- 4 Insertion before a given node.

$10 \rightarrow 20 \rightarrow 30$

can become:

$5 \rightarrow 10 \rightarrow 20 \rightarrow 30$

or:

$10 \rightarrow 20 \rightarrow 25 \rightarrow 30$

Insertion at the Beginning

Suppose we want to insert **5** before the current first node.

Two important steps:

$newNode \rightarrow next = START$

then:

$START = newNode$

This operation does not require traversal.

Therefore:

$O(1)$

Before

$START \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow NULL$

After

$START \rightarrow \boxed{5} \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow NULL$

Insertion at Beginning: C Code

```
struct Node *insertBeginning(  
    struct Node *start, int value)  
{  
    struct Node *newNode;  
  
    newNode = (struct Node *)  
        malloc(sizeof(struct Node));  
  
    newNode->data = value;  
  
    newNode->next = start;  
  
    start = newNode;  
  
    return start;  
}
```

Insertion at the End

Before

To insert a node at the end:

① Create a new node.

② Set:

$newNode \rightarrow next = NULL$

③ Traverse until the last node.

④ Connect the last node to the new node.

Without a separate tail pointer:

$O(n)$

$10 \rightarrow 20 \rightarrow 30 \rightarrow NULL$

New node:

$40 \rightarrow NULL$

After

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow NULL$

Insertion at End: C Code

```
struct Node *insertEnd(  
    struct Node *start, int value)  
{  
    struct Node *newNode, *ptr;  
  
    newNode = (struct Node *)  
        malloc(sizeof(struct Node));  
  
    newNode->data = value;  
    newNode->next = NULL;  
  
    if (start == NULL)  
        return newNode;  
  
    ptr = start;  
  
    while (ptr->next != NULL)
```

Insertion After a Given Node

Suppose we want to insert **25** after the node containing 20.

First locate the node containing 20.

Then perform:

$$newNode \rightarrow next = PTR \rightarrow next$$

followed by:

$$PTR \rightarrow next = newNode$$

The order is important because the existing link must not be lost.

Before

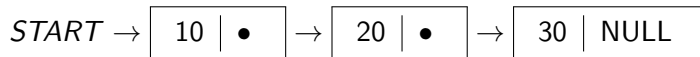
10 → 20 → 30

Insert 25 after 20:

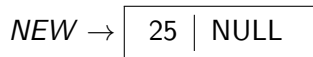
10 → 20 → 25 → 30

Pointer Changes During Insertion

Before insertion:



New node:



Step 1:

$NEW \rightarrow next = PTR \rightarrow next$

Step 2:

Insertion Before a Given Node

Suppose we want to insert **15** before 20.

We need two pointers:

PREPTR

and

PTR

where:

- PTR points to the target node.
- PREPTR points to the previous node.

Then:

PREPTR $\rightarrow$ *next* = *newNode*

Before

10 $\rightarrow$ 20 $\rightarrow$ 30

After

10 $\rightarrow$ 15 $\rightarrow$ 20 $\rightarrow$ 30

Deletion from a Singly Linked List

Common cases:

- 1 Delete the first node.
- 2 Delete the last node.
- 3 Delete a specific node.
- 4 Delete a node after a given node.

Before deleting dynamically allocated nodes, we must eventually release memory using:

free()

Attempting deletion from an empty list results in an underflow condition.

Deleting the First Node

Suppose:

$START \rightarrow 10 \rightarrow 20 \rightarrow 30$

To delete 10:

- 1 Save the first node:

$PTR = START$

- 2 Move START:

$START = START \rightarrow next$

- 3 Free the old node:

$free(PTR)$

Before

$START \rightarrow \boxed{10} \rightarrow 20 \rightarrow 30$

After

$START \rightarrow 20 \rightarrow 30 \rightarrow NULL$

Delete from Beginning: C Code

```
struct Node *deleteBeginning(  
    struct Node *start)  
{  
    struct Node *ptr;  
  
    if (start == NULL)  
    {  
        printf("UNDERFLOW\n");  
        return NULL;  
    }  
  
    ptr = start;  
    start = start->next;  
  
    free(ptr);  
  
    return start;  
}
```

Deleting the Last Node

To delete the last node, we need:

PTR

to point to the current node, and:

PREPTR

to point to the previous node.

When *PTR* reaches the last node:

$PREPTR \rightarrow next = NULL$

then:

$free(PTR)$

10 → 20 → $\underbrace{30}_{PREPTR}$ → $\underbrace{40}_{PTR}$ → *NULL*

After deletion:

10 → 20 → 30 → *NULL*

Without a tail pointer:

Delete from End: C Code

```
struct Node *deleteEnd(struct Node *start)
{
    struct Node *ptr, *preptr;

    if (start == NULL)
        return NULL;

    if (start->next == NULL)
    {
        free(start);
        return NULL;
    }

    ptr = start;

    while (ptr->next != NULL)
    {
```

Deleting a Specific Node

Suppose we want to delete 20:

$10 \rightarrow 20 \rightarrow 30$

We keep two pointers:

$PREPTR \rightarrow 10$

$PTR \rightarrow 20$

Then bypass the target node:

$PREPTR \rightarrow next = PTR \rightarrow next$

Finally:

$free(PTR)$

Before

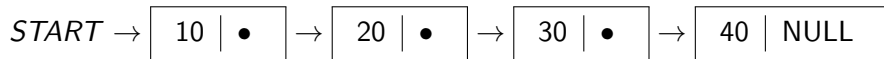
$10 \rightarrow \boxed{20} \rightarrow 30 \rightarrow 40$

After

$10 \rightarrow 30 \rightarrow 40 \rightarrow NULL$

Deletion by Pointer Bypass

Before:



Let:

$PREPTR \rightarrow 10$ $PTR \rightarrow 20$

Change:

$PREPTR \rightarrow next = PTR \rightarrow next$

After:

Underflow Condition

If:

$START = NULL$

then the linked list contains no nodes.

Trying to delete a node in this state results in:

UNDERFLOW

Therefore every deletion function should first check whether the list is empty.

$START \longrightarrow NULL$

No node exists

Time Complexity of Common Operations

Operation	Best Case	Worst Case
Traversal	$O(n)$	$O(n)$
Search	$O(1)$	$O(n)$
Insert at beginning	$O(1)$	$O(1)$
Insert at end	$O(n)$	$O(n)$
Delete beginning	$O(1)$	$O(1)$
Delete end	$O(n)$	$O(n)$
Insert after known node	$O(1)$	$O(1)$

Important

If the required node must first be searched, the search cost must also be included.

Complete Program: Structure and Display

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 void display(struct Node *start)
11 {
12     struct Node *ptr = start;
13
14     while (ptr != NULL)
15     {
16         printf("%d -> ", ptr->data);
17         ptr = ptr->next;
18     }
19 }
```

Complete Program: Insert Beginning

```
struct Node *insertBeginning(  
    struct Node *start, int value)  
{  
    struct Node *newNode;  
  
    newNode = (struct Node *)  
        malloc(sizeof(struct Node));  
  
    if (newNode == NULL)  
    {  
        printf("Memory allocation failed\n");  
        return start;  
    }  
  
    newNode->data = value;  
    newNode->next = start;
```

Complete Program: Insert End

```
struct Node *insertEnd(  
    struct Node *start, int value)  
{  
    struct Node *newNode, *ptr;  
  
    newNode = (struct Node *)  
        malloc(sizeof(struct Node));  
  
    newNode->data = value;  
    newNode->next = NULL;  
  
    if (start == NULL)  
        return newNode;  
  
    ptr = start;  
  
    while (ptr->next != NULL)
```

Complete Program: Delete Beginning

```
struct Node *deleteBeginning(  
    struct Node *start)  
{  
    struct Node *temp;  
  
    if (start == NULL)  
    {  
        printf("List is empty\n");  
        return NULL;  
    }  
  
    temp = start;  
    start = start->next;  
  
    free(temp);  
  
    return start;  
}
```

Complete Program: Main Function

```
int main()
{
    struct Node *start = NULL;

    start = insertBeginning(start, 30);
    start = insertBeginning(start, 20);
    start = insertBeginning(start, 10);

    printf("Linked List:\n");
    display(start);

    start = insertEnd(start, 40);

    printf("\nAfter inserting 40:\n");
    display(start);

    start = deleteBeginning(start);
```

Expected Output

Linked List:

10 -> 20 -> 30 -> NULL

After inserting 40:

10 -> 20 -> 30 -> 40 -> NULL

After deleting first node:

20 -> 30 -> 40 -> NULL

10 → 20 → 30



Why is `free()` Important?

Memory obtained using:

malloc()

remains allocated until it is explicitly released.

After removing a node from the linked list, call:

free(pointer)

Otherwise unused memory may remain allocated.

This is commonly called a:

Memory Leak

Before:

10 → 20 → 30

After bypass:

10 → 30

Old node 20 must be freed.

Classroom Activity

Consider:

$START \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow NULL$

Perform the following operations:

- 1 Insert 5 at the beginning.
- 2 Insert 25 after 20.
- 3 Delete 30.
- 4 Insert 50 at the end.

Draw the linked list after every operation.

Classroom Activity: Answer

Initial:

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$

Insert 5:

$5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40$

Insert 25:

$5 \rightarrow 10 \rightarrow 20 \rightarrow 25 \rightarrow 30 \rightarrow 40$

Delete 30:

$5 \rightarrow 10 \rightarrow 20 \rightarrow 25 \rightarrow 40$

Insert 50:

$5 \rightarrow 10 \rightarrow 20 \rightarrow 25 \rightarrow 40 \rightarrow 50$

Think Before You Code

Suppose:

$$PTR \rightarrow 20$$

in the list:

$$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$$

What does:

$$PTR = PTR \rightarrow next$$

do?

Think Before You Code

Suppose:

$$PTR \rightarrow 20$$

in the list:

$$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$$

What does:

$$PTR = PTR \rightarrow next$$

do?

It moves PTR from node 20 to node 30.

$PTR \rightarrow 30$

Find the Error

```
struct Node *newNode;  
  
newNode->data = 50;  
newNode->next = NULL;
```

What is wrong?

Find the Error

```
struct Node *newNode;  
  
newNode->data = 50;  
newNode->next = NULL;
```

What is wrong?

Memory has not been allocated for `newNode`.

Correct first step:

```
newNode = malloc(sizeof(struct Node));
```

MCQ 1

Which field stores the address of the next node?

- ① DATA
- ② START
- ③ NEXT
- ④ NULL

MCQ 1

Which field stores the address of the next node?

- ① DATA
- ② START
- ③ NEXT
- ④ NULL

NEXT

If:

START = NULL

then:

- ① The first node contains zero.
- ② The linked list is empty.
- ③ The last node has been deleted.
- ④ NEXT contains zero.

If:

START = NULL

then:

- ① The first node contains zero.
- ② The linked list is empty.
- ③ The last node has been deleted.
- ④ NEXT contains zero.

The linked list is empty

MCQ 3

What is the time complexity of inserting a node at the beginning?

- ① $O(n^2)$
- ② $O(n)$
- ③ $O(\log n)$
- ④ $O(1)$

MCQ 3

What is the time complexity of inserting a node at the beginning?

- ① $O(n^2)$
- ② $O(n)$
- ③ $O(\log n)$
- ④ $O(1)$

$O(1)$

What marks the end of a singly linked list?

- ① START
- ② HEAD
- ③ NULL
- ④ PTR

What marks the end of a singly linked list?

- ① START
- ② HEAD
- ③ NULL
- ④ PTR

NULL

Quick Recap

Node

DATA | NEXT

First node

START

Last node

NEXT = NULL

Traversal

$PTR = PTR \rightarrow next$

Insertion

Create + Connect

Deletion

Bypass + Free

Practice Questions

- 1 Write a C program to create and display a singly linked list.
- 2 Write a function to count the number of nodes.
- 3 Write a function to search for a given value.
- 4 Insert a node at the beginning and at the end.
- 5 Insert a node after a given value.
- 6 Delete the first and last nodes.
- 7 Delete a node containing a specified value.

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
- ② Reema Thareja, *Data Structures Using C*.
- ③ Horowitz, Sahni and Anderson-Freed, *Fundamentals of Data Structures in C*.

Your Feedback Matters

Please share your feedback about:

- Node and pointer visualization
- Traversal and searching
- Insertion operations
- Deletion operations
- C programming examples



Scan to Submit Feedback

Thank You

Questions?

START → Node → Node → *NULL*